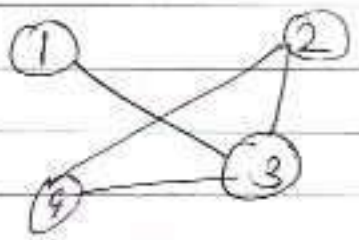
- Feedback form
- Viva on 10th

Tutorial - 8

- Undirected Graph $G = (V, E)$

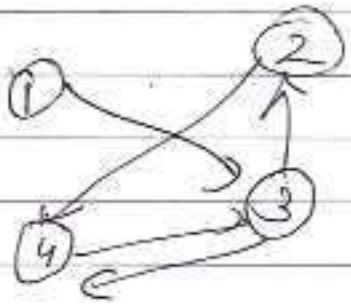
V = set of vertices
E = set of edges



$deg(4) = 2$

$N(4) = \{2, 3\} \subseteq V$

- Directed Graph
  $G = (V, E)$



$Deg_{in}(4) = 2$
$Deg_{out}(4) = 1$
$N_{in}(4) = \{2, 3\}$
$N_{out}(4) = \{3\}$

Rep: Adjacency matrix — Adj List

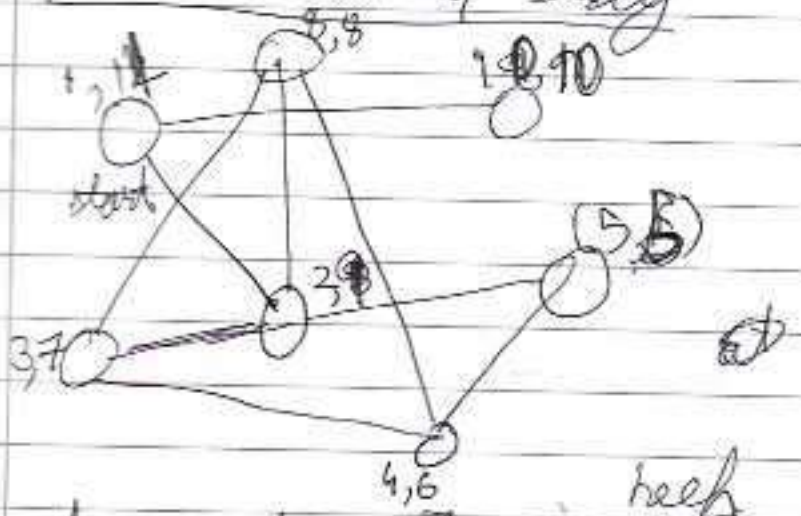
edge methods $O(1)$ — $O(deg(V) + deg(V))$ $O(deg(V))$

neighbors $O(n)$ — $O(n + m)$

space $O(n^2)$

How to explore a graph?

DFS: chalk & sleigh



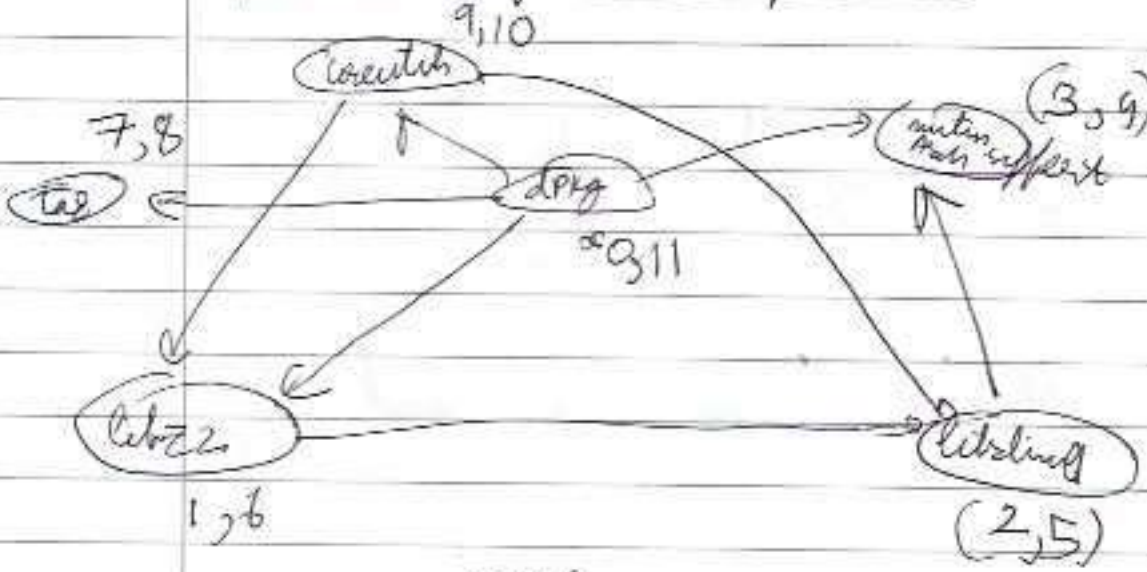
at each vertex, we ~~set~~ a time
of when it was entered + when we exited.

there is also an iterative version
using stack (think, ½ ~2 min)

```
DFS(u, time):
    u.start = time
    time += 1
    for v in u.neighbors
        if v is unvisited
            time = DFS(v, time)
            time += 1
    v.fin = time
    v = DEQUEUE
```

content: $O(m+n)$ ($\sum O(\deg(v)) + O(1)) = O(E+V)$)

Applications of DFS: topo sort



Do a DFS!

Can the start or end-times give us a plan?

suppose the graph has no cycles, (why?)

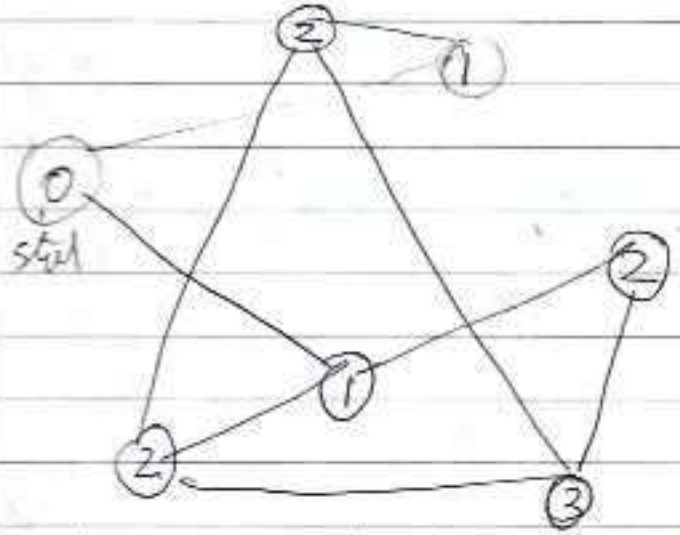
claim: A → B

if $t_{fA} > t_{fB}$

(simple case analysis + tireless)

So topsol = reverse order of finish time!
↳ push vertex as all done, push to stack!

BFS: Exploring with a breadth wise

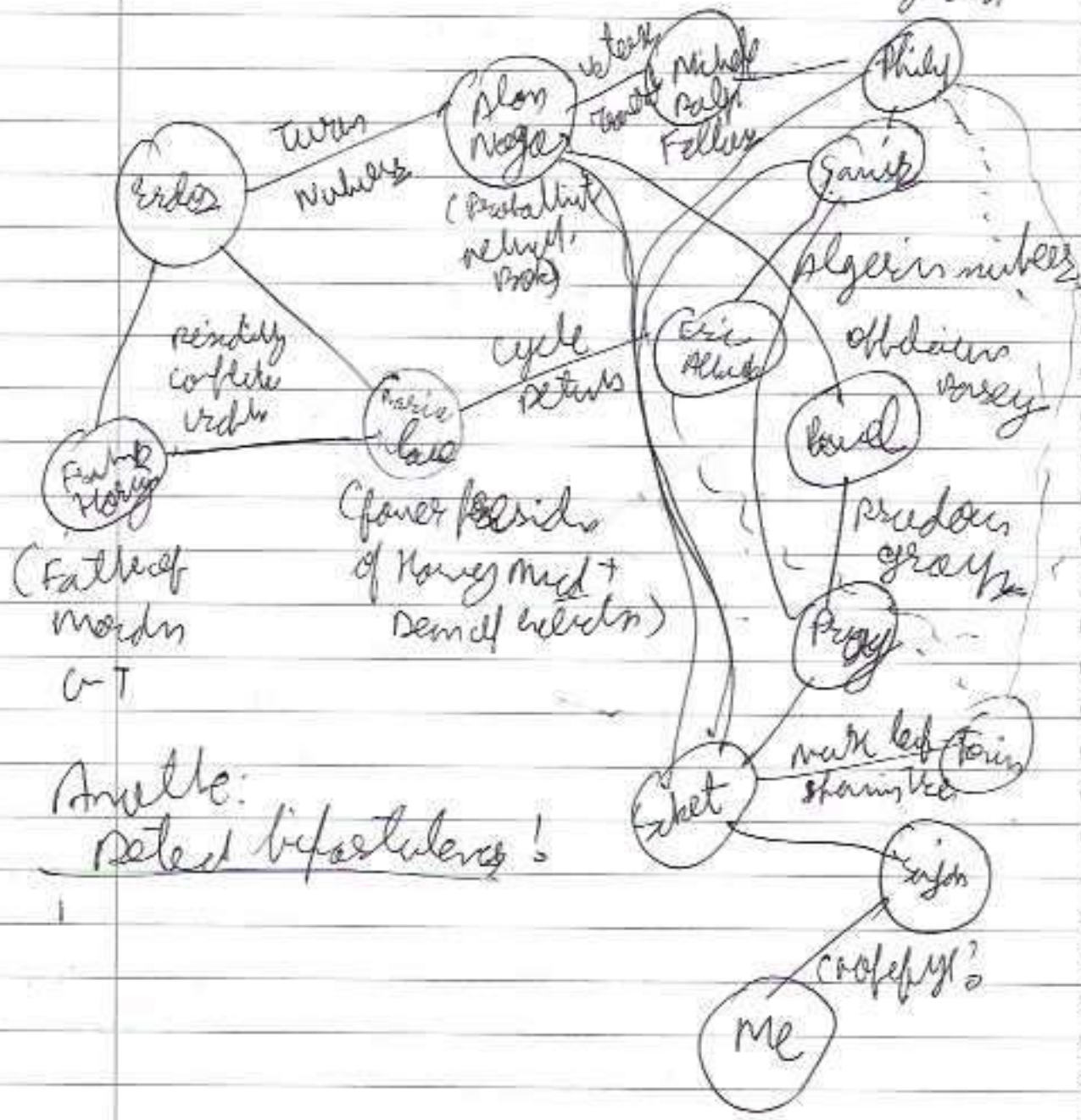


Set $L_i = []$ for $i = 1 \ldots n$
$L_0 = [s]$ where s is start
mark s as both
For $i = 0 \ldots n-1$
  For u in L_i
    For all v which is neighbour of u:
      if v is not explored:
        mark v as explored and put into L_{i+1}

L_i = nodes we explored in step i

Time: $O(m+n)$

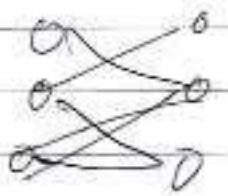
Uses of BFS: Find all connected components
shortest distance in unweighted graphs

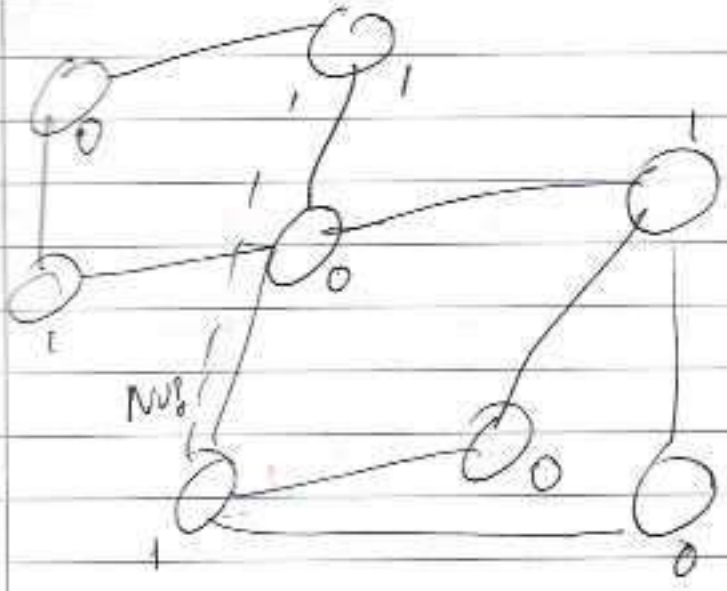


Answer:
Rated bi/astuteness!

Recall Karate club example!

Bipartite:



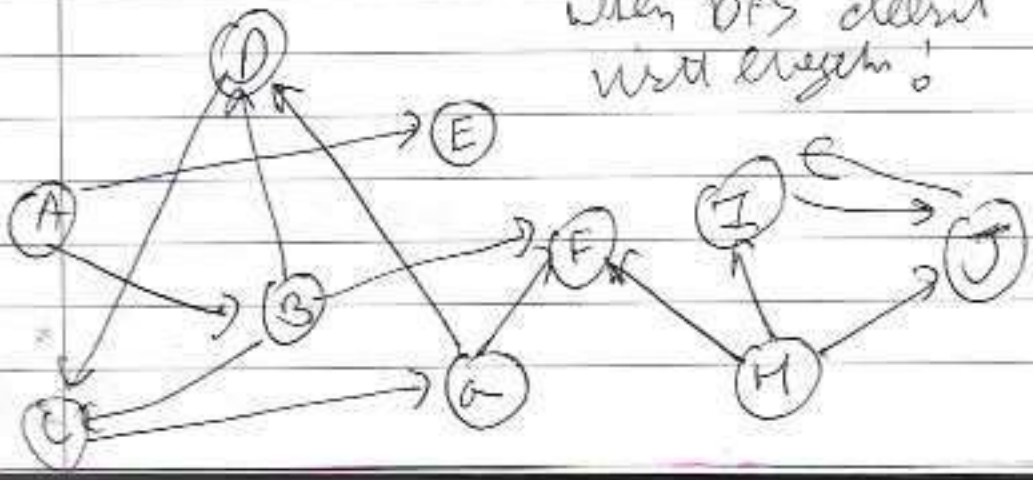


Idea:
BFS tree, color alternate levels

Claim: if 2 neighbors have same color, odd cycle in graph!

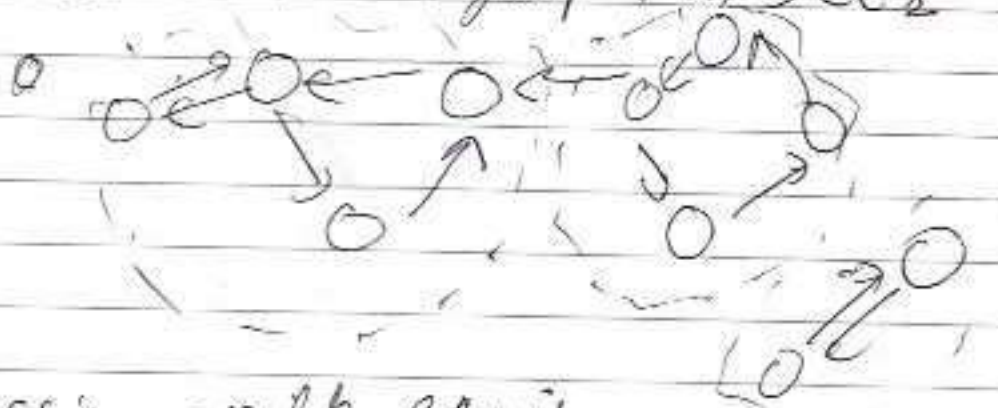
Claim: Odd cycle can't be 2-colored

Let's talk about SCC's!



SCC: for all u, v in V:
path from $u \to v$
path from $v \to u$
} SCC

we can decompose a graph in SCCs



SCC's are the equiv. classes under the relation

very useful to find commutables!

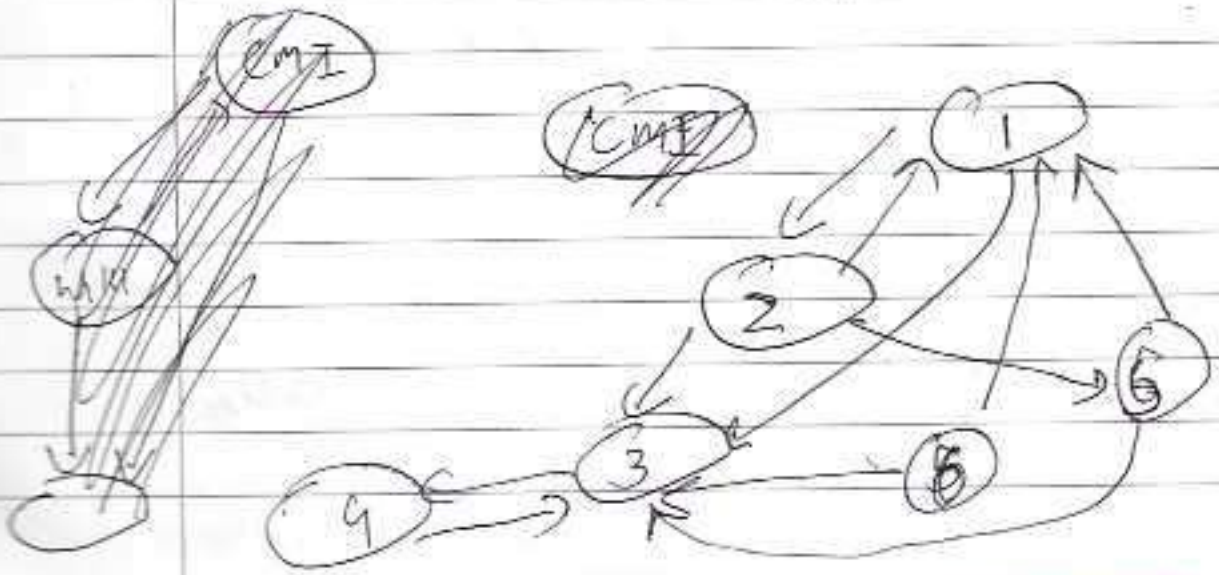
Naive algorithm:
SCCs = []
for each u:
Run DFS from u
For v reachable from u:
If v not in SCC we've found:
Run DFS from v to see if
also u is reachable
else: new SCC

Naive SCC: SCC[] = []

$\Omega(n^2)$

```
for each u:
    Run DFS from u
    For v reachable by u
        if v ∈ some SCC
            if u reachable by v:
                add u to v's SCC
            Else
                New SCC only u.
```

Kosaraju: Do DFS & create Forest (leaf nodes of end time)
Reverse all the edges
Do DFS again with order in terms of finish time
Different trees in new DFS are SCCs



why does this work?

Claim: if every SCC was a vertex, induced graph would be Directed Acyclic graph

Defn: Topological ordering of SCCs; vertices of all ~~edges~~ vertices brought of all vertices

Fact 1 11 11 :

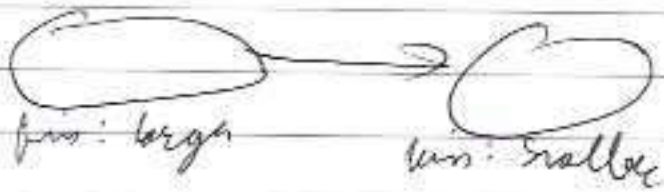
Idea: on reversing edges, SCC with fewer parents turn into have out Edges!

If I run DFS this will be entry of exit.

Reverse and repeat.

Pseudocode!

Claim: in SCC dag



11

Proof: Case 1: We reach A before B

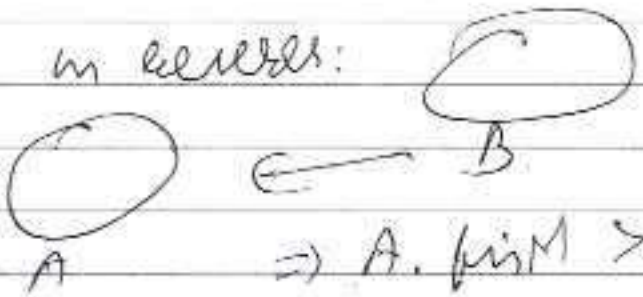
Say $y \in B$ has largest fin
$z \in A$ as discover fin

Ie: reach A before B
$\Rightarrow$ disc y via z
$\Rightarrow$ y is descen of z in DFS tree
$\Rightarrow$ A. fin > B. fins

Case 2: We reach B before A
- Use old no path from B to A
(then SCC graph is DAG)

~~Contrary~~ $\Rightarrow$ we finish exploring B and can't go to A
$\Rightarrow$ A isn't explored later!

Ex: in reverse:



$\Rightarrow$ A. finish > B. finish

solution: 1. KCPC on G^T, reverse key DONE

2. ~~[illegible]~~ feedback from

And now, we can KILL by induction

myth: First t vertices found in nearly DFS are t SCCs with largest finish times

Base: t=0 (trivial)

~~[illegible]~~ Step: By hyp, let first t there by last finishing

Consider (t+1)st vertex, suppose next is x

~~[illegible]~~ if x ∈ A (SCC)

Done

In A, fin > B. fin ∀ vertices (as x has largest finish time, by choice)

- No edge from A in SCC DAG (corollary)

- DFS after x leaves A
  - Nothing more by recursion
  - Nothing less by choice of x